Operating systemcvcv

(SUBJECT)

Practical File

Submitted By:

Garima f

Roll no. f

20001001036 f

Sec : A f

BTech CSE 2nd Year

Submitted To:asx

Ms. Heena

Practical 1

WINDOWS 2000

INTRODUCTION:

Windows 2000 is an operating system that runs on high-end desktop PCs and servers. In this case study, we will examine various aspects of Windows 2000 operating system, starting with introduction, then exploring its architecture, processes, memory management, I/O, the file system, and security aspects. Windows is a modern operating system that runs on consumer and business desktop PCs and enterprise servers.

Windows 2000 Professional:

It is used for workstation and Notebook. The original name is Windows NT5.9 Workstation. It supports two balanced multiprocessors at the most. The minimum internal storage is 64MB and maximum 4GB.

Windows 2000 Server:

It is developed for the small enterprises server field. Its original name is WindowsNT5.0 Server. It supports 4 CPU on one computer at the most. The minimum internal storage is 128MB and maximum 4GB.

Windows 2000 Advanced Server:

It is developed for the large and medium enterprises server field. Its original name is Windows NT 5.0 Server Enterprise Edition. It supports 8 CPU at the most. The minimum internal storage is 128MB and maximum 8GB.

Windows 2000 Datacenter Server:

It is developed for the highest level and scalability, usability, and reliability of large enterprises or state organizations server field. It supports 8 or more CPU (the maximum 32 CPU). The minimum internal storage is 256MB and maximum 64GB. In addition, Microsoft has provided the limited version Windows 2000 Advanced Server Limited Edition in 2001. It is used for the pure 64 bit microprocessor of Intel IA-64.

Features of Windows 2000

Some of the significant features of Windows 2000 are:

• Support for FAT16, FAT32 and NTFS

• Increased uptime of the system and significantly fewer OS reboot scenarios

• Windows Installer tracks applications and recognizes and replaces missing components

• Protects memory of individual apps and processes to avoid a single app bringing the system down

• Encrypted File Systems protect sensitive data

• Secure Virtual Private Networking (VPN) supports tunneling in to private LAN over public Internet

• Personalized menus adapt to the way you work

• Multilingual version allows for User Interface and help to switch, based on logon

• Includes broader support for high-speed networking devices, including Native ATM and cable modems

• Supports Universal Serial Bus (USB) and IEEE 13910 for greater bandwidth devices.

Programming Windows 2000:

The core of the NT operating system is the NTOS kernel mode program which provides the traditional system-call interfaces upon which the rest of the operating system is built. In Windows, only programmers at Microsoft write the system call layer. The published user mode interfaces all belong to operating system personalities that are implemented using subsystems that run on top of NTOS layers.

The Native Application Programming Interface

Windows 2000 has a set of calls it can perform. These are implemented in NTOS executive layer that runs in kernel mode. They are used internally by lower-level programs as kernel mode device drivers. Most of the native NT calls operate on kernel mode objects of one kind or another, including files, processes, threads, pipes, semaphores and so on.

The Win32 Application programming Interface

The Win32 function calls are collectively called the Win32 AO. These interfaces are publicly disclosed and documented. They are implemented as library procedures that either wrap the native NT system calls used to get the work done or, in some cases, do the work right in user mode. Though the native NT APOs, are not published, most of the functionality they provide is accessible through the Win32 API. The existing Win32 calls

rarely change with the new releases of Windows, though many new functions are added to the API.

The Windows Registry

The root of the NT namespace is maintained in the kernel. Storage, such as file system volumes, is attached to the NT namespace. Since the NT namespace is constructed afresh every time the system boots, how does the system know about any specific details of the system configuration? The answer is that windows attaches a special kind of file system to the NT namespace, the file system is called the registry. The registry is divided into separate volumes called hives. When a Windows system boots, one particular hive named SYSTEM is loaded into memory by the same boot program that loads kernel and other boot files, such as boot drivers, from the boot volume.

The Windows keeps a great deal of crucial information in the SYSTEM hive, including information about drivers to use with what devices, what software run initially and many parameters governing the operation of the system.

Kernel

The kernel sits between the HAL and the Executive and provides multiprocessor synchronization, thread and interrupt scheduling and dispatching, and trap handling and exception dispatching; it is also responsible for initializing device drivers at bootup that are necessary to get the operating system up and running. That is, the kernel performs almost all the tasks of a traditional microkernel; the strict distinction between Executive and Kernel is the most prominent remnant of the original microkernel design, and historical design documentation consistently refers to the kernel component as “the microkernel”.

The kernel often interfaces with the process manager. The level of abstraction is such that the kernel never calls into the

process manager, only the other way around (save for a handful of corner cases, still never to the point of a functional dependence).

Kernel-mode drivers

Windows NT uses kernel-mode device drivers to enable it to interact with hardware devices. Each of the drivers has well defined system routines and internal routines that it exports to the rest of the operating system. All devices are seen by user mode code as a file object in the I/O manager, though to the I/O manager itself the devices are seen as device objects, which it defines as either file, device or driver objects. Kernel mode drivers exist in three levels: highest level drivers, intermediate drivers and low level drivers. The highest level drivers, such as file system drivers for FAT and NTFS, rely on intermediate drivers. Intermediate drivers consist of function drivers—or main driver for a device—that are optionally sandwiched between lower and higher level filter drivers. The function driver then relies on a bus driver—or a driver that services a bus controller, adapter, or bridge—which can have an optional bus filter driver that sits between itself and the function driver. Intermediate drivers rely on the lowest level drivers to function. The Windows Driver Model (WDM) exists in the intermediate layer. The lowest level drivers are either legacy Windows NT device drivers that control a device directly or can be a PnP hardware bus. These lower level drivers directly control hardware and do not rely on any other drivers.

Hardware abstraction layer

The Windows NT hardware abstraction layer, or HAL, is a layer between the physical hardware of the computer and the rest of the operating system. It was designed to hide differences in hardware and therefore provide a consistent platform on which the kernel is run. The HAL includes hardware-specific code that controls I/O interfaces, interrupt controllers and multiple processors.

However, despite its purpose and designated place within the architecture, the HAL isn't a layer that sits entirely below the kernel, the way the kernel sits below the Executive: all known HAL implementations depend in some measure on the kernel, or even the Executive. In practice, this means that kernel and HAL variants come in matching sets that are specifically engineered to work together.

In particular hardware abstraction does not involve abstracting the instruction set, which generally falls under the wider concept of portability. Abstracting the instruction set, when necessary (such as for handling the several revisions to the x86 instruction set, or emulating a missing math coprocessor), is performed by the kernel, or via platform virtualization.

Summary:

Kernel mode in Windows 2000 is structure in the HAL, the kernel and executive layers of NTSOS, and a large number of device drivers implementing everything from device services to file systems and networking to graphics. The HAL hides certain differences in hardware from other components. The kernel layer manages the CPUs to support multithreading and synchronization and the executive implements the most kernel mode services.

The executive is based on kernel mode objects that represent the key executive live data structures, including processes, threads, memory sections, drivers, devices and synchronization objects.

The important objects in Windows are processes, threads and sections. Processes have virtual address spaces and are containers for resources. Threads are the unit of execution and are scheduled by the kernel layer using a priority algorithm in which highest priority ready thread always runs, preempting lower priority as necessary. Sections represent memory objects, like files, that can be mapped into the address spaces of processes. EXE and DLL program images are represented as sections, as is shared memory.

Windows supports virtual memory. The paging algorithm is based in the working set concept. The system maintains several types of page lists, to optimize the use of memory. The cache manager manages virtual addresses in the kernel that can be used to map files into memory, dramatically improving I/O performance for many applications because read operations can be satisfied without accessing disk.

I/O is performed by device drivers, which follow the Windows Driver Model. Each driver starts out by initializing a driver object that contains the addresses of the procedures that the system can call to manipulate devices. Devices are stacked and I/O request packets are passed down the stack and services by the drivers for each device stack.

Finally, Windows 2000 has a sophisticated security system. The security infrastructure in Windows 2000 features Kerberos Authentication Protocol. Windows 2000 security integrates the centralized information store and policy-based control provided by Active Directory with industry standard protocols for cross-platform, secure connections between clients and servers. In addition, Windows 2000 includes integrated the public key infrastructure components to support advanced security features such as smart cards for two-factor authentication and EFS for secure on-disk data storage.

Practical – 2

Administration of WINDOWS 2000(including DNS , LDAP , Directory Services , Difference between windows and Linux).

Introduction

Make computers easier and more comfortable to use with accessibility features and utilities built right into Microsoft windows 2000(professional and home versions).Display and readability the job description for a systems administrator says that little creativity is required, but this is not necessarily the case. Often, the job requires a great deal of creative thinking in order to solve a problem quickly. If a systems problem crops up, there may be more than one ”fix,” and it is the job of the systems administrator to determine which solution will be the most effective. He may also be in charge of upgrading the system and deciding which software is the most effective for the job it does. Generally, all these works done by administrator are part of system administrator are part of system administration.

Directory services

What Is a Directory Service?

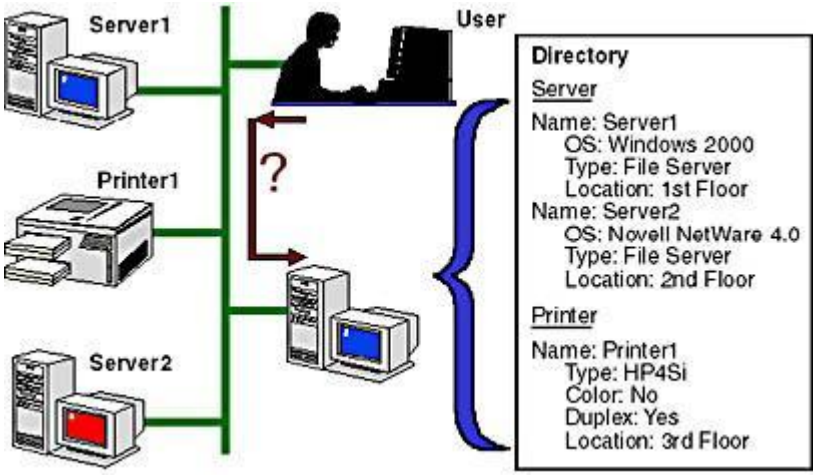
A *directory* is a stored collection of information about objects that are related to one another in some way. For example, a telephone directory

stores names of entities and their corresponding telephone numbers. The telephone directory listing may also contain an address or other information about the entity.

In a distributed computing system or a public computer network such as the Internet, there are many objects, such as file servers, printers, fax servers, applications, databases, and users. Users must be able to locate and use these objects. Administrators must be able to manage how these objects are used. A directory service stores all the information needed to use and manage these objects in a centralized location, simplifying the process of locating and managing these resources.

Why Have a Directory Service?

A directory service provides the means to organize and simplify access to resources of a networked computer system. Users and administrators may not know the exact name of the objects they need. However, they may know one or more attributes of the objects in question. As illustrated in the figure below, they can use a directory service to query the directory for a list of objects that match known attributes. For example, "Find all color printers on the third floor" queries the directory for all color printer objects with the attributes of color and third floor (or maybe a location attribute that has been set to "third floor"). A directory service makes it possible to find an object based on one or more of its attributes.



Using a directory service

Other functions of the directory service are:

- Enforcing security to protect the objects in its database from outside intruders or from internal users who do not have permission to access those objects.
- Distributing a directory across many computers in a network.
- Replicating a directory to make it available to more users and resistant to failure.
- Partitioning a directory into multiple stores that are located on different computers across the network. This makes more space available to the directory as a whole and allows the storage of a large number of objects.

A directory service is both an administration tool and an end user tool. As a network becomes larger, more resource objects must be managed and the directory service becomes a necessity.

Windows 2000 Directory Services

Active Directory is the directory service included in Windows 2000 Server. Active Directory includes the directory, which stores information about network resources, as well as all the services that make the information available and useful. The resources stored in the directory, such as user data, printers, servers, databases, groups, services, computers, and security policies, are known as *objects.*

Active Directory is integrated within Windows 2000 Server and offers:

- Simplified administration
- Scalability
- Open standards support
- Support for standard name formats

Simplified Administration

Active Directory organizes resources hierarchically in domains. A *domain* is a logical grouping of servers and other network resources under a single domain name. The domain is the basic unit of replication and security in a Windows 2000 network.

Each domain includes one or more domain controllers. A *domain controller* is a computer running Windows 2000 Server that manages user access to a network, which includes logging on, authentication, and access to the directory and shared resources. To simplify administration, all domain controllers in the domain are equal. You can make changes to any

domain controller, and the updates are replicated to all other domain controllers in the domain.

Scalability

In Active Directory, the directory stores information by organizing the directory into sections that permit storage for a very large number of objects. As a result, the directory can expand as an organization grows, allowing users to scale from a small installation with a few hundred objects to a large installation with millions of objects.

DNS

Domain Name System (DNS) is one of the industry-standard suite of protocols that comprise TCP/IP, and together the DNS Client and DNS Server provide computer name-to-IP address mapping name resolution services to computers and users.

Support for LDAP and HTTP

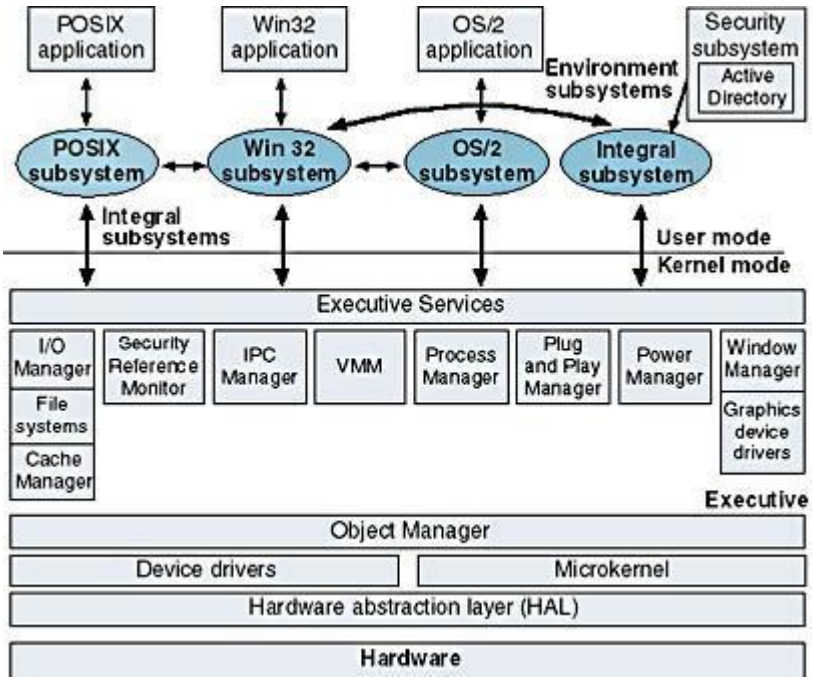
Active Directory further embraces Internet standards by directly supporting LDAP and HTTP. LDAP is a version of the X.500 directory access protocol, which was developed as a simpler alternative to the Directory Access Protocol (DAP). Active Directory supports both LDAP version 2 and version 3. HTTP is the standard protocol for displaying pages on the World Wide Web. A user can display every object in Active Directory as an HTML page in a Web browser. Thus, users receive the benefit of the familiar Web browsing model when querying and viewing objects in Active Directory.

Active Directory in the Windows 2000 Architecture

Windows 2000 uses modules and modes that combine to provide operating system services to applications. Two processor access modes, *kernel* and *user*, divide the low-level, platform-specific processes from the upper level processes, respectively, to shield applications from platform differences and to prevent direct access to system code and data by applications. Each application, including service applications, runs in a separate *module* in user mode, from which it requests system services through an API that gains limited access to system data. An application process begins in user mode and is transferred to kernel mode, where the actual service is provided in a protected environment. The process is then transferred back to user mode. Active Directory runs in the security subsystem in user mode. The *security reference monitor*, which runs in kernel mode, is the primary authority for enforcing the security rules of the

security subsystem. The following figure shows the location of Active Directory within Windows 2000.

The tight integration of the directory service and security subsystem services is key to the implementation of Windows 2000 distributed systems. Access to all directory objects first requires proof of identity (authentication), which is performed by components of the security subsystem, and then validation of access permissions (authorization), which is performed by the security subsystem in conjunction with the security reference monitor. The security reference monitor enforces the access control applied to Active Directory objects.



The key service components include the following:

- **Directory System Agent (DSA)** builds a hierarchy from the parent-child relationships stored in the directory. Provides APIs for directory access calls.
- **Database Layer** provides an abstraction layer between applications and the database. Calls from applications are never made directly to the database; they go through the database layer.
- **Extensible Storage Engine** communicates directly with individual records in the directory data store on the basis of the object's relative distinguished name attribute.
- **Data store (the database file NTDS.DIT)** is manipulated only by the Extensible Storage Engine database engine, stored in the \Winnt\ntds folder on the domain controller. You can administer the

file by using the Ntdsutil tool, located in the \Winnt\system32 folder on the domain controller.

Clients obtain access to Active Directory by using one of the following mechanisms that is supported by the DSA:

- **LDAP/ADSI.** Clients that support LDAP use it to connect to the DSA. Active Directory supports LDAP version 3 (defined by RFC 2251) and LDAP version 2 (defined by RFC 1777). Windows 2000 clients, as well as Windows 98 and Windows 95 clients that have the Active Directory client components installed, use LDAP version 3 to connect to the DSA. ADSI is a means of abstracting LDAP API; however, Active Directory uses only LDAP.
- **Messaging API (MAPI).** Legacy MAPI clients, such as Microsoft Outlook, connect to the DSA by using the MAPI RPC address book provider interface.
- **Security Accounts Manager (SAM).** Windows clients that use Windows NT 4.0 or earlier use the SAM interface to connect to the DSA. Replication from backup domain controllers in a mixed-mode domain goes through the SAM interface as well.
- **Replication (REPL).** When they are performing directory replication, Active Directory DSAs connect to each other by using a proprietary RPC interface.

Difference between WINDOWS and LINUX

Linux:

Linux could be a free and open supply OS supported operating system standards. It provides programming interface still as programme compatible with operating system primarily based systems and provides giant selection applications. A UNIX operating system system additionally contains several severally developed parts, leading to UNIX operating system that is totally compatible and free from proprietary code.

Windows:

Windows may be a commissioned OS within which ASCII text file is inaccessible. it's designed for the people with the angle of getting no programming information and for business and alternative industrial users. it's terribly straightforward and simple to use.

The distinction between Linux and Windows package is that Linux is completely freed from price whereas windows is marketable package and is expensive. Associate operating system could be a program meant to

regulate the pc or computer hardware Associate behave as an treater between user and hardware.

Linux is a open supply package wherever users will access the ASCII text file and might improve the code victimisation the system. On the opposite hand, in windows, users can't access ASCII text file, and it's a authorized OS.

S.NO	Linux	Windows
1.	Linux is a open source operating system.	While windows are the not the open source operating system.
2.	Linux is free of cost.	While it is costly.
3.	It's file name case-sensitive.	While it's file name is case-insensitive.
4.	In linux, monolithic kernel is used.	While in this, micro kernel is used.
5.	Linux is more efficient in comparison of windows.	While windows are less efficient.
6.	There is forward slash is used for Separating the directories.	While there is back slash is used for Separating the directories.
7.	Linux provides more security than windows.	While it provides less security than linux.
8.	Linux is widely used in hacking purpose based systems.	While windows does not provide much efficiency in hacking.

Practical – 3

Study of LINUX Operating System (Linux kernel , shell , basic commands , pipe and filter commands).

Linux

- Linux is an open source operating system (OS). An operating system is the software that directly manages a system's hardware and resources, like CPU, memory, and storage.
- The OS sits between applications and hardware and makes the connections between all of your software and the physical resources that do the work.

WORKING OF LINUX

- Think about an OS like a car engine. An engine can run on its own, but it becomes a functional car when it's connected with a transmission, axles, and wheels. Without the engine running properly, the rest of the car won't work.
- The OS includes some common core components, like the GNU tools, among others.

- These tools give the user a way to manage the resources provided by the kernel, install additional software, configure performance and security settings, and more.
- All of these tools bundled together make up the functional operating system. Because Linux is an open source OS, combinations of software can vary between Linux distributions.

Kernel

The Linux kernel is the main component of a Linux operating system (OS) and is the core interface between a computer's hardware and its processes.The kernel is so named because—like a seed inside a hard shell—it exists within the OS and controls all the major functions of the hardware, whether it's a phone, laptop, server, or any other kind of computer.

What the kernel does?

The kernel has 4 jobs:

1. **Memory management:** Keep track of how much memory is used to store what, and where
2. **Process management:** Determine which processes can use the central processing unit (CPU), when, and for how long
3. **Device drivers:** Act as mediator/interpreter between the hardware and processes
4. **System calls and security:** Receive requests for service from the processes

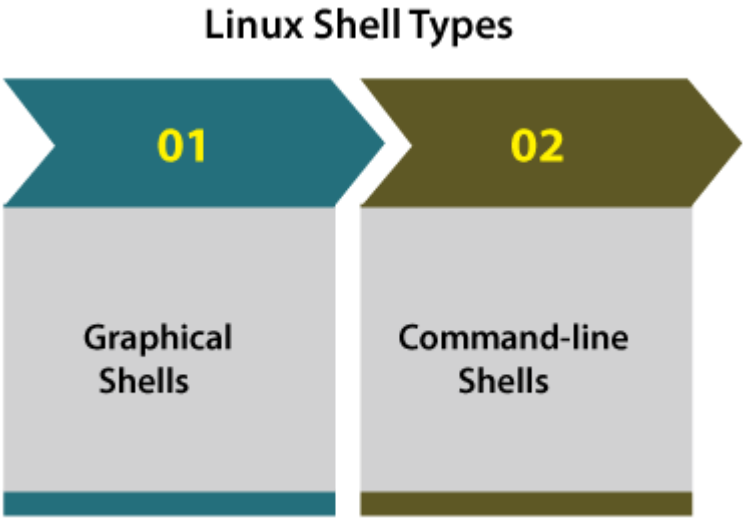
Linux Shell

So, basically, a shell is a program that receives commands from the user and gives it to the OS to process, and it shows the output. Linux's shell is its main part. Its distros come in GUI (graphical user interface), but basically, Linux has a CLI (command line interface). In this tutorial, we are going to cover the basic commands that we use in the shell of Linux.

To open the terminal, press Ctrl+Alt+T in Ubuntu, or press Alt+F2, type in gnome-terminal, and press enter. In Raspberry Pi, type in lxterminal.

C . Shell –

An interface to kernel, hiding complexity of kernel's functions from users. The shell takes commands from the user and executes kernel's functions.



Basic Linux Commands

pwd

The **pwd** command stands for **print working directory**. It tells the user the location and name of the current directory (that is the directory where the user is working currently), likewise doesn't accept arguments. Instead, it gets its input through the shell.

```
garima@garima-VirtualBox:~$ pwd
/home/garima
garima@garima-VirtualBox:~$ S
```

man

The man stands for manual. The man command displays the user manual of any command that we run on the terminal.

ls

The **ls** is the list command in Linux. It will show the full list or content of your directory. Just type *ls* and press the enter key.

```
garima@garima-VirtualBox:~$ ls
2d      d2  d8          file1.sh  g5.sh    Music     sample3     x1
abecho  d3  d9          fruits    garima1  myfile1   snap        x2
d1      d4  Desktop     g         gimi     Pictures  Templates   x3
d11     d5  Documents   g1.txt    gimu     Public    vegetables  x4
d13     d6  Downloads   g3.sh     himu     sample    Videos      xz
d14     d7  file1.sg    g4.sh     hrdik    sample1   x
garima@garima-VirtualBox:~$
```

Ls – a : The (ls -a) command will enlist the whole list of the current directory including the hidden files.

```
garima@garima-VirtualBox:~$ ls -a
.                d6          gimu        .sudo_as_admin_successful
..               d7          .gnupg      Templates
2d               d8          himu        .vboxclient-clipboard.pid
abecho           d9          hrdik       .vboxclient-display-svga-x11.pid
.bash_history    Desktop     .local      .vboxclient-draganddrop.pid
.bash_logout     Documents   .mozilla    .vboxclient-seamless.pid
.bashrc          Downloads   Music       vegetables
.cache           file1.sg    myfile1     Videos
.config          file1.sh    Pictures    x
d1               fruits      .profile    x1
d11              g           Public      x2
d13              g1.txt      .s          x3
d14              g3.sh       sample      x4
d2               g4.sh       sample1     xz
d3               g5.sh       sample3     .y
d4               garima1     snap
d5               gimi        .ssh
```

Ls – i: It will show the list in a long list format.

```
garima@garima-VirtualBox:~$ ls -i
180239 2d       180235 d8          131205 g5.sh      131191 sample3
180259 abecho   180236 d9          180229 garima1    180260 snap
180245 d1       399601 Desktop     180237 gimi       399603 Templates
180238 d11      399605 Documents   131179 gimu       131181 vegetables
180240 d13      399602 Downloads   131180 himu       399608 Videos
180241 d14      136223 file1.sg    180258 hrdik      180242 x
180246 d2       136224 file1.sh    399606 Music      180247 x1
180230 d3       180228 fruits      131178 myfile1    180248 x2
180231 d4       131171 g           399607 Pictures   180249 x3
180232 d5       131206 g1.txt      399604 Public     180250 x4
180233 d6       131201 g3.sh       131190 sample     180244 xz
180234 d7       131204 g4.sh       131174 sample1
```

Ls – r : It is used to print the list in reverse order.

```
garima@garima-VirtualBox:~$ ls -r
xz  Videos       sample    himu     g3.sh     Downloads  d6   d13
x4  vegetables   Public    gimu     g1.txt    Documents  d5   d11
x3  Templates    Pictures  gimi     g         Desktop    d4   d1
x2  snap         myfile1   garima1  fruits    d9         d3   abecho
x1  sample3      Music     g5.sh    file1.sh  d8         d2   2d
x   sample1      hrdik     g4.sh    file1.sg  d7         d14
```

Ls – t : It will sort the list by displaying recently modified filed at top.

```
garima@garima-VirtualBox:~$ ls -t
y1          g1.txt    sample    2d          myfile1   d8          Public
y2          g5.sh     garima1   himu        xz        d9          Templates
y3          g4.sh     x4        x           d13       d4          Videos
practical   abecho    x1        vegetables  d14       d3
file1.sh    hrdik     x2        fruits      d11       sample1
file1.sg    g3.sh     x3        gimu        d5        Desktop
snap        gimi      d1        Documents   d6        Downloads
Pictures    sample3   d2        g           d7        Music
```

mkdir

The mkdir command can do a few other things with directories as well. The mkdir command can set the permissions for a specified directory or multiple directories.

Creating 1 directory :

```
garima@garima-VirtualBox:~$ mkdir practical
garima@garima-VirtualBox:~$ cd practical
garima@garima-VirtualBox:~/practical$
```

Creating multiple directories:

```
garima@garima-VirtualBox:~$ mkdir y{1..3}
garima@garima-VirtualBox:~$ ls
2d      d3  Desktop    g1.txt   himu       sample      x   y2
abecho  d4  Documents  g3.sh    hrdik      sample1     x1  y3
d1      d5  Downloads  g4.sh    Music      sample3     x2
d11     d6  file1.sg   g5.sh    myfile1    snap        x3
d13     d7  file1.sh   garima1  Pictures   Templates   x4
d14     d8  fruits     gimi     practical  vegetables  xz
d2      d9  g          gimu     Public     Videos      y1
```

rmdir

The rmdir command removes each directory specified on the command line, if they are empty. That is, each directory removed must contain no files or directories, or it cannot be removed by rmdir.

```
garima@garima-VirtualBox:~$ rmdir y{1..3}
garima@garima-VirtualBox:~$ ls
2d      d2  d8          file1.sh  g5.sh    Music      sample1     x
abecho  d3  d9          fruits    garima1  myfile1    sample3     x1
d1      d4  Desktop     g         gimi     Pictures   snap        x2
d11     d5  Documents   g1.txt    gimu     practical  Templates   x3
d13     d6  Downloads   g3.sh     himu     Public     vegetables  x4
d14     d7  file1.sg    g4.sh     hrdik    sample     Videos      xz
garima@garima-VirtualBox:~$
```

cat

cat (concatenate) command is very frequently used in Linux. It reads data from the file and gives their content as output. It helps us to create, view, concatenate files.

```
garima@garima-VirtualBox:~$ cat d8
cat: d8: Is a directory
```

rm

rm command in UNIX stands for remove and by default is used for removing files. It is simple but a powerful command especially when used with options such as -rf which allow it to delete non-empty directories forcefully. The rm command, By default, cannot remove Directories and only works on files.

- **rm -i** will ask before deleting each file. Some people will have rm aliased to do this automatically (type "alias" to check). Consider using **rm -I** instead, which will only ask once and only if you are trying to delete three or more files.
- **rm -r** will recursively delete a directory and all its contents (normally rm will not delete directories, while rmdir will only delete empty directories).
- **rm -f** will forcibly delete files without asking; this is mostly useful if you have rm aliased to ``rm -i'' but want to delete lots of files without confirming each one.

Touch

It is used to create a file without any content. The file created using touch command is empty. This command can be used when the user doesn't have data to store at the time of file creation.

```
garima@garima-VirtualBox:~$ touch a.txt
```

Chmod

the **chmod** command is used to change the access mode of a file.

```
garima@garima-VirtualBox:~$ chmod u+x a.txt
garima@garima-VirtualBox:~$ ls -l
total 172
drwxrwxr-x 2 garima garima 4096 Apr  5 14:25 2d
drwxrwxr-x 2 garima garima 4096 Apr 20 03:46 abecho
-rwxrw-r-- 1 garima garima    0 May 11 00:03 a.txt
```

Head

The head command, as the name implies, print the top N number of data of the given input .

```
garima@garima-VirtualBox:~$ cat > 1
garima
gimi
garima
gimu
gimmu
garima@garima-VirtualBox:~$ head -3 1
garima
gimi
garima
```

Tail

It is the complementary of head command.The tail command, as the name implies, print the last N number of data of the given input. By default it prints the last 10 lines of the specified files. If more than one file name is provided then data from each file is precedes by its file name.

```
garima@garima-VirtualBox:~$ tail -3 1
garima
gimu
gimmu
```

Help

The **help** command which as its name says help you to learn about any built-in command.

```
garima@garima-VirtualBox:~$ help
GNU bash, version 5.0.17(1)-release (x86_64-pc-linux-gnu)
These shell commands are defined internally.  Type `help' to see this list.
Type `help name' to find out more about the function `name'.
Use `info bash' to find out more about the shell in general.
Use `man -k' or `info' to find out more about commands not in this list.

A star (*) next to a name means that the command is disabled.

 job_spec [&]                         history [-c] [-d offset] [n] or >
 (( expression ))                     if COMMANDS; then COMMANDS; [ el>
 . filename [arguments]               jobs [-lnprs] [jobspec ...] or j>
 :                                    kill [-s sigspec | -n signum | ->
 [ arg... ]                           let arg [arg ...]
 [[ expression ]]                     local [option] name[=value] ...
 alias [-p] [name[=value] ... ]       logout [n]
 bg [job_spec ...]                    mapfile [-d delim] [-n count] [->
 bind [-lpsvPSVX] [-m keymap] [-f >   popd [-n] [+N | -N]
 break [n]                            printf [-v var] format [argument>
 builtin [shell-builtin [arg ...]>    pushd [-n] [+N | -N | dir]
 caller [expr]                        pwd [-LP]
 case WORD in [PATTERN [| PATTERN]>   read [-ers] [-a array] [-d delim>
 cd [-L|[-P [-e]] [-@]] [dir]         readarray [-d delim] [-n count] >
 command [-pVv] command [arg ...]     readonly [-aAf] [name[=value] ..>
 compgen [-abcdefgjksuv] [-o optio>   return [n]
 complete [-abcdefgjksuv] [-pr] [->   select NAME [in WORDS ... ;] do >
 compopt [-o|+o option] [-DEI] [na>   set [-abefhkmnptuvxBCHP] [-o opt>
 continue [n]                         shift [n]
 coproc [NAME] command [redirectio>   shopt [-pqsu] [-o] [optname ...]
 declare [-aAfFgilnrtux] [-p] [nam>   source filename [arguments]
 dirs [-clpv] [+N] [-N]               suspend [-f]
 disown [-h] [-ar] [jobspec ... | >   test [expr]
 echo [-neE] [arg ...]                time [-p] pipeline
 enable [-a] [-dnps] [-f filename]>   times
```

```
echo [-neE] [arg ...]                time [-p] pipeline
enable [-a] [-dnps] [-f filename]>   times
eval [arg ...]                       trap [-lp] [[arg] signal_spec ..>
exec [-cl] [-a name] [command [ar>   true
exit [n]                             type [-afptP] name [name ...]
export [-fn] [name[=value] ...] o>   typeset [-aAfFgilnrtux] [-p] nam>
false                                ulimit [-SHabcdefiklmnpqrstuvxPT>
fc [-e ename] [-lnr] [first] [las>   umask [-p] [-S] [mode]
fg [job_spec]                        unalias [-a] name [name ...]
for NAME [in WORDS ... ] ; do COM>   unset [-f] [-v] [-n] [name ...]
for (( exp1; exp2; exp3 )); do CO>   until COMMANDS; do COMMANDS; do>
function name { COMMANDS ; } or n>   variables - Names and meanings o>
getopts optstring name [arg]         wait [-fn] [id ...]
hash [-lr] [-p pathname] [-dt] [n>   while COMMANDS; do COMMANDS; do>
help [-dms] [pattern ...]            { COMMANDS ; }
garima@garima-VirtualBox:~$
```

Exit

exit command in linux is used to exit the shell where it is currently running. It takes one more parameter as *[N]* and exits the shell with a return of status *N*. If n is not provided, then it simply returns the status of last command that is executed.

More

more command is used to view the text files in the command prompt, displaying one screen at a time in case the file is large (For example log files). The more command also allows the user do scroll up and down through the page.

Less

Less command is a Linux utility that can be used to read the contents of a text file one page(one screen) at a time.

Piping

A pipe is a form of redirection (transfer of standard output to some other destination) that is used in Linux and other Unix-like operating systems to send the output of one command/program/process to another command/program/process for further processing. The Unix/Linux systems

allow stdout of a command to be connected to stdin of another command. You can make it do so by using the pipe character '|'.
Pipe is used to combine two or more commands, and in this, the output of one command acts as input to another command, and this command's output may act as input to the next command and so on. It can also be visualized as a temporary connection between two or more commands/ programs/ processes.

```
garima@garima-VirtualBox:~$ ls -l | more
total 176
-rw-rw-r-- 1 garima garima  126 May 11 00:16 1
drwxrwxr-x 2 garima garima 4096 Apr  5 14:25 2d
drwxrwxr-x 2 garima garima 4096 Apr 20 03:46 abecho
-rwxrw-r-- 1 garima garima    0 May 11 00:03 a.txt
drwxrwxr-x 2 garima garima 4096 Apr  5 14:34 d1
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d11
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d13
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d14
drwxrwxr-x 2 garima garima 4096 Apr  5 14:34 d2
drwxrwxr-x 2 garima garima 4096 Mar 29 14:21 d3
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d4
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d5
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d6
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d7
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d8
drwxrwxr-x 2 garima garima 4096 Mar 29 14:22 d9
drwxr-xr-x 2 garima garima 4096 Mar 28 10:02 Desktop
drwxr-xr-x 5 garima garima 4096 Mar 29 15:19 Documents
drwxr-xr-x 2 garima garima 4096 Mar 28 10:02 Downloads
-rw-rw-r-- 1 garima garima    0 Apr 27 02:53 file1.sg
-rw-rw-r-- 1 garima garima   69 Apr 27 03:09 file1.sh
drwxrwxr-x 2 garima garima 4096 Mar 29 15:57 fruits
-rw-rw-r-- 1 garima garima    8 Mar 29 14:39 g
-rw-rw-r-- 1 garima garima   14 Apr 20 04:41 g1.txt
-rwxrwxr-x 1 garima garima   18 Apr 20 03:40 g3.sh
-rw-rw-r-- 1 garima garima  215 Apr 20 04:10 g4.sh
-rw-rw-r-- 1 garima garima  233 Apr 20 04:40 g5.sh
drwxrwxr-x 2 garima garima 4096 Apr  6 02:45 garima1
drwxrwxr-x 2 garima garima 4096 Apr  6 02:53 gimi
```

Practical – 4

Administration of LINUX Operating System.

Linux system administration is a process of setting up, configuring, and managing a computer system in a Linux environment. System administration involves creating a user account, taking reports, performing backup, updating configuration files, documentation, and performing recovery actions. The user who manages the server, fixes configuration issues, recommends new software updates, and updates document is the **system administrator**.

Linux operating system interface

The Linux operating system offers two types of user interface, CLI and GUI. The CLI is mostly used on the server-side, while GUI is used on the client-side. The interface depends on what the user intends to achieve with the Linux OS.

The Linux OS serves as the backbone for network infrastructure, Cloud computing, web servers, database servers, email servers, and directory service. Each of these servers and services requires a system administrator to manage them.

System administration is carried out by an expert in the IT (Information Technology) industry. The expert ensures that all computing devices and networks work properly. The job demand depends on the kind of system, software application, and work environment.

System administrator allows the administrator to allocate user privileges based on the required resources and function of the user. The system administrator is

the most privileged user, known as the *super user*, and determines what computing resources each user receives.

Creating a group in LINUX

1 . To create a new group, use the **groupadd** command.

Type the following command:

```
groupadd -g group-ID group-name
```

where *group-ID* is the numeric identifier of the group, and *group-name* is

1. To add a member to a supplementary group, use the **usermod** command to list the supplementary groups that the user is currently a member of, and the supplementary groups that the user is to become a member of.

   For example, if the user is already a member of the group groupa, and is to become a member of groupb, use the following command:

   ```
   usermod -G groupa,groupb user-name
   ```

   where *user-name* is the user name.

2. To display who is a member of a group, use the **getent** command.

   Type the following command:

   ```
   getent group group-name
   ```

   where *group-name* is the name of the group.

3. To remove a member from a supplementary group, use the **usermod** command to list the supplementary groups that you want the user to remain a member of.

   For example, if the user's primary group is users and the user is also a member of the groups mqm, groupa and groupb, to remove the user from the mqm group, use the following command:

   ```
   usermod -G groupa,groupb user-name
   ```

where *user-name* is the user name.

Creating a new user

The Basic syntax of the **useradd** command is:

```
# useradd [options] username
```

When we add a new user in Linux with the '**useradd**' command it gets created in a locked state and to unlock that user account, we need to set a password for that account with the '**passwd**' command.

```
[root@tecmint ~]# passwd tecmint

Changing password for user tecmint.

New UNIX password:

Retype new UNIX password:

passwd: all authentication tokens updated successfully.
```

Once a new user is created, its entry is automatically added to the '**/etc/passwd**' file. The file is used to store the user's information and the entry should be.

```
tecmint:x:1000:1000:tecmint:/home/tecmint:/bin/bash
```

Top command

The **top** (**t**able **o**f **p**rocesses) command shows a real-time view of running processes in Linux and displays kernel-managed tasks. The command also provides a system information summary that shows resource utilization, including CPU and memory usage.

Ps command

ps command is **used to list the currently running processes and their PIDs along with some other information depends on different options**. It reads the process information from the virtual files in /proc file-system. /proc contains virtual files, this is the reason it's referred as a virtual file system.

Kill command

In Linux, the kill command (located inside the /bin/kill) can be defined as a built-in command. It is **used for manually terminating the processes**. The behaviour of the kill command is slightly different among the shells and the /bin/kill standalone executable.

Practical 5

A) What is shell script? Explain the steps to write and run shell script .Explain sha-bang .

Answer: A sequence of commands saved to a file and this file is nothing but shell script .

Inside shell script, we can also use programming features like conditional statements , loops, functions etc, Hence we can write scripts more easily for complex requirements also.

- **Why do we need shell scripts:**There are many reasons to write shell scripts –
    - To avoid repetitive work and automation
    - System admins use shell scripting for routine backups
    - System monitoring
    - Adding new functionality to the shell etc.
- **Advantages of shell scripts**
    - The command and syntax are exactly the same as those directly entered in command line, so programmer do not need to switch to entirely different syntax
    - Writing shell scripts are much quicker
    - Quick start
    - Interactive debugging etc.
- **Disadvantages of shell scripts**
    - Prone to costly errors, a single mistake can change the command which might be harmful
    - Slow execution speed
    - Design flaws within the language syntax or implementation
    - Not well suited for large and complex task

- Provide minimal data structure unlike other scripting languages. Etc.

HOW TO WRITE AND RUN SHELL SCRIPTS:

a) Write script
-save the file name with .sh extension and using cat > command write within the script

b) Provide execution permission to the script:
chmod a+x file name with .sh extension

c) Run the script:
1)/bin/bash ./file name with.sh extension
2)bash ./filename.sh

Sha -Bang:

The shebang is the combination of the # (pound key) and ! (exclamation mark). This character combination has a special meaning when it is used in the very first line of the script. It is used to specify the interpreter with which the given script will be run by default.

So, if the first line of a script is:

#!/bin/bash

b) Write a script to display present working directory, current date and calendar.

Code: - practical5.sh

```
#! /bin/bash
date
cal
pwd
```

Output: -

```
garima@hardik-virtualbox:~$ /bin/bash ./practical5.sh
Tuesday 17 May 2022 02:42:11 PM IST
      May 2022
Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7
 8  9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30 31

/home/garima
```

c)Write a script to read student name, roll number, subject, fee and save to a file “out.txt”.

Code: - input file= bp.sh, output file=out.txt

```
read -p “enter your name:” name
read -p “enter your roll number:” number
read -p “enter subject:” subject
read -p “enter your fee:” fee
echo “Student name: $name”
echo “Student roll number: $number”
echo “Student subject: $subject”
echo “Student fee: $fee”
echo “Student name: $name” >>out.txt
echo “Student roll number: $number” >>out.txt
echo “Student subject: $subject” >>out.txt
echo “Student fee: $fee” >>out.txt
```

Output:

```
garima@hardik-virtualbox:~$ /bin/bash ./bp.sh
Enter your name:Gimu
Enter your roll number:36
Enter subject :OS
Enter your fee:45000
 Student name : Gimu
 Student roll number : 36
 Student subject : OS
 Student fee : 45000
garima@hardik-virtualbox:~$ cat out.txt
 Student name : Gimu
 Student roll number : 36
 Student subject : OS
 Student fee : 45000
```

d)Write a script to print table of a number entered through console.

Code: - practical5.sh

```
#! /bin/bash
read -p “enter the number:” n
i=1
while [ $i -le 10]
```

```
do
echo " $n * $i = $((i*n))"
let i++
done
```

Output:

```
garima@hardik-virtualbox:~$ /bin/bash ./condition.sh
enter the number :4
 4 * 1 = 4
 4 * 2 = 8
 4 * 3 = 12
 4 * 4 = 16
 4 * 5 = 20
 4 * 6 = 24
 4 * 7 = 28
 4 * 8 = 32
 4 * 9 = 36
 4 * 10 = 40
```

e. Write a script to read input from console sort them and print uniq.

Code : sort.sh

```
cat >> sort.sh
echo $1
echo $2
echo $3
echo $4
echo $5
echo $6
-> bash ./ sort.sh anu aisha garima deepika harsh garima | sort
-> bash ./ sort.sh anu aisha garima deepika harsh garima | sort | uniq
```

Output :

```
garima@hardik-virtualbox:~$ bash ./sort.sh anu aisha garima deepika harsh garima | sort
aisha
anu
deepika
garima
garima
harsh
garima@hardik-virtualbox:~$ bash ./sort.sh anu aisha garima deepika harsh garima | sort | uniq
aisha
anu
deepika
garima
harsh
garima@hardik-virtualbox:~$
```

Practical 6

Write a program to implement FCFS algorithm.

```
#include<iostream>

using namespace std;

int main()

{   int n,bt[20],wt[20],tat[20],avwt=0,avtat=0,i,j;

    cout<<"Enter total number of processes(maximum 20):";

    cin>>n;


    cout<<"\nEnter Process Burst Time aka DURATION \n";

    for(i=0;i<n;i++)

    {

        cout<<"P["<<i+1<<"]:";

        cin>>bt[i];

    }

    wt[0]=0;    //waiting time for first process is 0
```

```
    //calculating waiting time
    for(i=1;i<n;i++)
    {
        wt[i]=0;
        for(j=0;j<i;j++)
            wt[i]+=bt[j];
    }
    cout<<"\nProcess\t\tBurst Time\tWaiting Time\tTurnaround Time";
    //calculating turnaround time
    for(i=0;i<n;i++)
    {
        tat[i]=bt[i]+wt[i];
        avwt+=wt[i];
        avtat+=tat[i];
        cout<<"\nP["<<i+1<<"]"<<"\t\t"<<bt[i]<<"\t\t"<<wt[i]<<"\t\t"<<tat[i];
    }
    avwt/=i;
    avtat/=i;
    cout<<"\n\nAverage Waiting Time:"<<avwt;
    cout<<"\nAverage Turnaround Time:"<<avtat;


    return 0;
}
```

Solution:

```
Enter total number of processes(maximum 20):2

Enter Process Burst Time aka DURATION
P[1]:2
P[2]:1

Process         Burst Time      Waiting Time    Turnaround Time
P[1]            2               0               2
P[2]            1               2               3

Average Waiting Time:1
Average Turnaround Time:2
PS D:\CODING\C>
```

Practical 7

Write a program to implement SJF algorithm.

Code:

```
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;

typedef  struct proccess
{
    int at,bt,ct,ta,wt,btt;
    string pro_id;

        /*
    artime = Arrival time,
    bt = Burst time,
    ct = Completion time,
    ta = Turn around time,
    wt = Waiting time
    */

}Schedule;

bool compare(Schedule a,Schedule b)
{
    return a.at<b.at;
```

```
    /* This Schedule will always return TRUE
    if above condition comes*/
}
bool compare2(Schedule a,Schedule b)
{
    return a.bt<b.bt;

    /* This Schedule will always return TRUE
    if above condition comes*/
}

int main()
{
    Schedule pro[10];
    //An array of Processes
    int n,i,j,pcom;
    //n = number of processes, i= iteration variable

    cout<<"Enter the number of Process::";
    cin>>n;

    cout<<"Enter the Process id arrival time burst time :::";

    for(i=0;i<n;i++)
    {
        cin>>pro[i].pro_id;
        cin>>pro[i].at;
        cin>>pro[i].bt;
        pro[i].btt=pro[i].bt;
    }

    sort(pro,pro+n,compare);

    i=0;
    pcom=0;
    while(pcom<n)
    {
        for(j=0;j<n;j++)
        {
            if(pro[j].at>i)
            break;
        }

        sort(pro,pro+j,compare2);

        if(j>0)
        {
```

```
            for(j=0;j<n;j++)
            {
                if(pro[j].bt!=0)
                break;
            }
            if(pro[j].at>i)

            {
                i=pro[j].at;

            }
            pro[j].ct=i+1;
            pro[j].bt--;
        }
        i++;
        pcom=0;
        for(j=0;j<n;j++)
        {
            if(pro[j].bt==0)
            pcom++;
        }
    }

    cout<<"ProID\tAtime\tBtime\tCtime\tTtime\tWtime\n";

    for(i=0;i<n;i++)
    {
        pro[i].ta=pro[i].ct-pro[i].at;
        pro[i].wt=pro[i].ta-pro[i].btt;

cout<<pro[i].pro_id<<"\t"<<pro[i].at<<"\t"<<pro[i].btt<<"\t"<<pro[i].ct<<"\t"<
<pro[i].ta<<"\t"<<pro[i].wt;
        cout<<endl;
    }
    return 0;
}
```

Solution:

```
Enter the number of Process::2
Enter the Process id arrival time burst time :::
1 2 3
2 0 5
ProID   Atime   Btime   Ctime   Ttime   Wtime
2       0       5       5       5       0
1       2       3       8       6       3
PS D:\CODING\C>
```

PRACTICAL 8

Write a program to implement round robin scheduling algorithm.

Code:

```
#include<iostream>
using namespace std;
// Function to find the waiting time for all
// processes
void findWaitingTime(int processes[], int n,
            int bt[], int wt[], int quantum)
{

    int rem_bt[n];
    for (int i = 0 ; i < n ; i++)
        rem_bt[i] = bt[i];
    int t = 0; // Current time

    while (1)
    {
        bool done = true;

        for (int i = 0 ; i < n; i++)
        {
            if (rem_bt[i] > 0)
            {
                done = false; // There is a pending process
                if (rem_bt[i] > quantum)
                {
```

```
                    t += quantum;

                    rem_bt[i] -= quantum;
                }
                else
                {
                    t = t + rem_bt[i];
                    wt[i] = t - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }

        if (done == true)
        break;
    }
}

void findTurnAroundTime(int processes[], int n,
                        int bt[], int wt[], int tat[])
{

    for (int i = 0; i < n ; i++)
        tat[i] = bt[i] + wt[i];
}
// Function to calculate average time
void findavgTime(int processes[], int n, int bt[],
                                    int quantum)
{
    int wt[n], tat[n], total_wt = 0, total_tat = 0;
    findWaitingTime(processes, n, bt, wt, quantum);
    findTurnAroundTime(processes, n, bt, wt, tat);
    // Display processes along with all details
    cout << "Processes "<< " Burst time "
        << " Waiting time " << " Turn around time\n";
    for (int i=0; i<n; i++)
    {
        total_wt = total_wt + wt[i];
        total_tat = total_tat + tat[i];
        cout << " " << i+1 << "\t\t" << bt[i] <<"\t "
            << wt[i] <<"\t\t " << tat[i] <<endl;
    }
    cout << "Average waiting time = "
        << (float)total_wt / (float)n;
    cout << "\nAverage turn around time = "
        << (float)total_tat / (float)n;
}
int main()
{
```

```
    int processes[] = { 1, 2, 3};
    int n = sizeof processes / sizeof processes[0];
    int burst_time[] = {10, 5, 8};

    int quantum = 2;
    findavgTime(processes, n, burst_time, quantum);
    return 0;
}
```

Solution:

```
Processes  Burst time  Waiting time  Turn around time
 1              10        13              23
 2              5         10              15
 3              8         13              21
Average waiting time = 12
Average turn around time = 19.6667
```

PRACTICAL 9

Write a program to implement LRU page replacement.

Code:

```
#include<stdio.h>

int findLRU(int time[], int n){
int i, minimum = time[0], pos = 0;

for(i = 1; i < n; ++i){
if(time[i] < minimum){
minimum = time[i];
pos = i;
}
}
return pos;
}

int main()
{
   int no_of_frames, no_of_pages, frames[10], pages[30], counter = 0, time[10], flag1, flag2, i, j, pos, faults = 0;
printf("Enter number of frames: ");
scanf("%d", &no_of_frames);
printf("Enter number of pages: ");
scanf("%d", &no_of_pages);
printf("Enter reference string: ");
   for(i = 0; i < no_of_pages; ++i){
    scanf("%d", &pages[i]);
   }

for(i = 0; i < no_of_frames; ++i){
    frames[i] = -1;
   }

   for(i = 0; i < no_of_pages; ++i){
    flag1 = flag2 = 0;

    for(j = 0; j < no_of_frames; ++j){
```

```
    if(frames[j] == pages[i]){
    counter++;
    time[j] = counter;
  flag1 = flag2 = 1;
  break;
  }
    }

    if(flag1 == 0){
for(j = 0; j < no_of_frames; ++j){
    if(frames[j] == -1){
    counter++;
    faults++;
    frames[j] = pages[i];
    time[j] = counter;
    flag2 = 1;
    break;
    }
    }
    }

    if(flag2 == 0){
    pos = findLRU(time, no_of_frames);
    counter++;
    faults++;
    frames[pos] = pages[i];
    time[pos] = counter;
    }

    printf("\n");

    for(j = 0; j < no_of_frames; ++j){
    printf("%d\t", frames[j]);
    }
}
printf("\n\nTotal Page Faults = %d", faults);

   return 0;
}
```

Solution:

```
Enter number of frames: 3
Enter number of pages: 6
Enter reference string: 1 2 4 3 2 1

1       -1      -1
1       2       -1
1       2       4
3       2       4
3       2       4
3       2       1

Total Page Faults = 5
```

PRACTICAL - 10

Write a program to implement Banker's algorithm.

Code:

```
#include <stdio.h>
int main()
{

    int n, r, i, j, k;
    n = 5;
    r = 3;
    int alloc[5][3] = { { 0, 0, 1 },
                        { 3, 0, 0 },
                        { 1, 0, 1 },
                        { 2, 3, 2 },
                        { 0, 0, 3 } };
    int max[5][3] = { { 7, 6, 3 },
                    { 3, 2, 2 },
                    { 8, 0, 2 },
                    { 2, 1, 2 },
                    { 5, 2, 3 } };

    int avail[3] = { 2, 3, 2 };

    int f[n], ans[n], ind = 0;
    for (k = 0; k < n; k++) {
        f[k] = 0;
    }
    int need[n][r];
    for (i = 0; i < n; i++) {
        for (j = 0; j < r; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }
    int y = 0;
    for (k = 0; k < 5; k++) {
        for (i = 0; i < n; i++) {
```

```
            if (f[i] == 0) {

                int flag = 0;
                for (j = 0; j < r; j++) {
                    if (need[i][j] > avail[j]){
                        flag = 1;
                        break;
                    }
                }

                if (flag == 0) {
                    ans[ind++] = i;
                    for (y = 0; y < r; y++)
                        avail[y] += alloc[i][y];
                    f[i] = 1;
                }
            }
        }
    }

    printf("Th SAFE Sequence is as follows\n");
    for (i = 0; i < n - 1; i++)
        printf(" P%d ->", ans[i]);
    printf(" P%d", ans[n - 1]);

    return (0);

}
```

Solution:

```
Th SAFE Sequence is as follows
 P1 -> P3 -> P4 -> P0 -> P2
PS D:\CODING\C>
```

PRACTICAL 11

Awk programming:

A program is a solution to a problem, formulated in the syntax of a particular language. It is a small step from writing complex editing scripts with `sed` to writing programs with `awk`, but it is a step that many writers may fear to take. “Script” is less loaded a term than “program” for many people, but an editing script is still a program.

Each programming language has its own “style” that lends itself to performing certain tasks better than other languages. Anyone can

scan a reference page and quickly learn a language's syntax, but a close examination of programs written in that language is usually required before you understand how to apply this knowledge. In this sense, a programming language is simply another tool; you need to know not only how to use it but also when and why it is used.

The `awk` program was designed for text-processing applications, particularly those in which information is structured in records and fields. The major capabilities of `awk` that we will demonstrate in upcoming pages are as follows:

- definable record and field structure
- conditional and looping constructs
- assignment, arithmetic, relational, and logical operators
- numeric and associative arrays
- formatted print statements
- built-in functions

To increment the value of `x` by 1 using the shell, you'd use the following line:

```
x='expr $x + 1'
```

The `expr` command is a UNIX program that is executed as a separate process returning the value of its arguments. In `awk`, you only have to write:

```
++X
```

This is the same as `x = x + 1`. (This form could also be used in `awk`.)

▪ Invoking `awk` ▪

The `awk` program itself is a program that, like `sed`, runs a specified program on lines of input. You can enter `awk` from the command line, or from inside a shell script.

$ **awk** '*program*' *files*

Input is read a line at a time from one or more *files.* The *program,* enclosed in single quotation marks to protect it from the shell, consists of *pattern* and *procedure* sections. If the pattern is missing, the procedure is performed on all input lines:

`$ awk '{print}' sample`	*Prints all lines in* `sample` *file*

The procedure is placed within braces. If the procedure is missing, lines matching the pattern are printed:

`$ awk '/programmer's guide/' sample`	*Prints lines matching pattern in* `sample` *file*

The `awk` program allows you to specify zero, one, or two pattern addresses, just like `sed`. Regular expressions are placed inside a pair of slashes (/). In `awk`, patterns can also be made up of expressions. An expression (or a primary expression so as not to confuse it with a regular expression) can be a string or numeric constant (for example, *red* or *l*), a variable (whose value is a string or numeric), or a function (which we'll look at later).

You can associate a pattern with a specific procedure as follows:

```
/pattern1/  {
              procedure 1
         }
/pattern2/  {
              procedure 2
         }
{ procedure 3  }
```

Like `sed`, only the lines matching the particular pattern are the object of a procedure, and a line can match more than one pattern. In this example, the third procedure is performed on all input lines. Usually, multiline `awk` scripts are placed in a separate file and invoked using the `-f` option:

```
$ awk -f awkscript sample
```

▪ Records and Fields ▪

Perhaps the most important feature of `awk` is that it divides each line of input into fields. In the simplest case, each field contains a single word, delimited by a blank space. The `awk` program allows you to reference these fields by their position in the input line, either in patterns or procedures. The symbol `$0` represents the entire input line. `$1, $2`, . . . refer, by their position in the input line, to individual fields.

We'll demonstrate some of these capabilities by building an `awk` program to search through a list of acronyms in a file. Each acronym is listed along with its meaning. If we print the first field of each line, we'll get the name of the acronym:

```
$ awk '{print $1}' sample
BASIC
CICS
COBOL
DBMS
GIGO
GIRL
```

We can construct a useful program that would allow you to specify an acronym and get its description. We could use `awk` just like `grep`:

```
$ awk '/BASIC/' sample
BASIC Beginner's All-Purpose Symbolic Instruction Code
```

However, there are three things we'd like to do to improve this program and make better use of `awk's` capabilities:

1. Limit the pattern-matching search.
2. Make the program more general and not dependent on the particular acronym that is the subject of the search.
3. Print only the description.

▪ Testing Fields ▪

The pattern as specified will match the word *BASIC* anywhere on the line. That is, it might match *BASIC* used in a description. To see if the first field (`$1`) matches the pattern, we write:

```
$1  ==  "BASIC"
```

The symbol `==` is a relational operator meaning “equal to” and is used to compare the first field of each line with the string *BASIC.* You could also construct this test using a given regular expression that looks for the acronym at the beginning of the line.

```
$1 ~ /^BASIC/
```

The pattern-matching operator `~` evaluates as true if an expression `($1)` matches a regular expression. Its opposite, `!~`, evaluates true if the expression does not match the regular expression.
Although these two examples look very similar, they achieve very different results. The relational operator `==` evaluates true if the first field is *BASIC* but false if the first field is *BASIC,* (note the comma). The pattern-matching operator `~` locates both occurrences.

Pattern-matching operations must be performed on a regular expression (a string surrounded by slashes). Variables cannot be used inside a regular expression with the exception of shell variables, as shown in the next section. Constants cannot be evaluated using the pattern-matching operator.

▪ Passing Parameters from a Shell Script ▪

Our program is too specific and requires too much typing. We can put the `awk` script in a file and invoke it with the `-f` option. Or we can put the command inside a shell script, named for the function it performs. This shell script should be able to read the first argument from the command line (the name of the acronym) and pass it as a parameter to `awk`. We’ll call the shell script `awkronym` and set it up to read a file named `acronyms`. Here’s the simplest way to pass an argument into an `awk` procedure:

```
$ cat awkronym
awk '$1 == search' search=$1 acronyms
```

Parameters passed to an `awk` program are specified *after* the program. The `search` variable is set up to pass the first argument on

the command line to the `awk` program. Even this gets confusing, because `$1` inside the `awk` program represents the first field of each input line, while `$1` in the shell represents the first argument supplied on the command line. Here's how this version of the program works:

```
$ awkronym CICS
CICS Customer Information Control System
```

By replacing the search string `BASIC` with a variable (which could be set to the string `CICS` or `BASIC`), we have a program that is fairly generalized.

Notice that we had to test the parameter as a string (`$1 == search`). This is because we can't pass the parameter inside a regular expression. Thus, the expressions "`$1 ~ /search/`" or "`$1 ~ search`" will produce syntax errors.

As an aside, let's look at another way to import a shell variable into an `awk` program that even works inside a regular expression. However, it looks complicated:

```
search=$1
awk '$1 ~ /'"$search"'/' acronyms
```

▪ Changing the Field Separator ▪

The `awk` program is oriented toward data arranged in fields and records. A record is normally a single line of input, consisting of one or more fields. The field separator is a blank space or tab and the record separator is a newline. For example, here's one record with five fields:

```
CICS Customer Information Control System
```

Field three or `$3` is the string *Information.* In our program, we like to be able to print the description as a field. It is obvious that we can't just say `print $2` and get the entire description. But that is what we'd like to be able to do.

This will require that we change the input file using another character (other than a blank) to delimit fields. A tab is frequently used as a field separator. We'll have to insert a tab between the first and second fields:

```
$ cat acronyms
awk   Aho, Weinstein & Kernighan
BASIC Beginner's All-Purpose Symbolic Instruction Code
CICS  Customer Information Control System
COBOL Common Business Orientated Language
DBMS  Data Base Management System
GIGO  Garbage In, Garbage Out
GIRL  Generalized Information Retrieval Language
```

You can change the field separator from the command line using the `-F` option:

```
$ awk -F" |———————|" '$1 == search {print $2)'
search=$1 acronyms
```

Note that | ——— | is entered by typing a double quotation mark, pressing the *TAB* key, and typing a double quotation mark. This makes the tab character (represented in the example as | ——— |) the exclusive field separator; spaces no longer serve to separate fields. Now that we've implemented all three enhancements, let's see how the program works:

```
$ awkronym GIGO
Garbage In, Garbage Out
```

System Variables

The `awk` program defines a number of special variables that can be referenced or reset inside a program. See Table 13-1.

TABLE 13-1. awk System Variables

System Variable	Meaning
FILENAME	Current filename
FS	Field separator (a blank)
NF	Number of fields in the current record
NR	Number of the current record
OFS	Output field separator (a blank)
ORS	Output record separator (a newline)
RS	Record separator (a newline)

The system variable `FS` defines the field separator used by `awk`. You can set `FS` inside the program as well as from the command line.

Typically, if you redefine the field or record separator, it is done as part of a `BEGIN` procedure. The `BEGIN` procedure allows you to specify an action that is performed before the first input line is read.

```
BEGIN { FS = "|──────|" }
```

You can also specify actions that are performed after all input is read by defining an `END` procedure.
The `awk` command sets the variable `NF` to the number of fields on the current line. Try running the following `awk` command on any text file:

```
$ awk '{print $NF}' test
```

If there are five fields in the current record, `NF` will be set to five; `$NF` refers to the fifth and last field. Shortly, we'll look at a program, `double`, that makes good use of this variable.

▪ Looping ▪

The `awkronym` program can print field two because we restructured the input file and redefined the field separator. Sometimes, this isn't practical, and you need another method to read or print a number of fields for each record. If the field separator is a blank or tab, the two records would have six and five fields, respectively.

. For instance, if we want to perform a procedure three times, we keep track of how many times we go through the loop by incrementing a variable at the bottom of the loop, then we check at the top of the loop to see if that variable is greater than 3. Let's take an example in which we print the input line three times.

```
{       i = 1
        while (i <= 3) {
               print
               ++i
               }
        }
```

Braces are required inside the loop to describe a procedure consisting of more than a single action. Three operators are used in this program: `=` assigns the value 1 to the variable `i`; `<=` compares the value of `i` to the constant 3; and `++` increments the variable by 1. The first time the `while` statement is encountered, `i` is equal to 1.

Because the expression `i <= 3` is true, the procedure is performed. The last action of the procedure is to increment the variable `i`. The `while` expression is true after the end of the second loop has incremented `i` to 3. However, the end of the third loop increments `i` to 4 and the expression evaluates as false.
A `for` loop serves the same purpose as a `while` loop, but its syntax is more compact and easier to remember and use. Here's how the previous `while` statement is restructured as a `for` loop:

```
for (i = 1; i <= 3; i++)
      print
```

The `for` statement consists of three expressions within parentheses. The first expression, `i = 1`, sets the initial value for the counter variable. The second expression states a condition that is tested at the top of the loop. (The `while` statement tested the condition at the bottom of the loop.) The third expression increments the counter.
Now, to loop through remaining fields on the line, we have to determine how many times we need to execute the loop. The system variable `NF` contains the number of fields on the current input record. If we compare our counter (`i`) against `NF` each time through the loop, we'll be able to tell when all fields have been read:

```
for (i = 1; i <= NF; i++)
```

We will print out each field (`$i`), one to a line. Just to show how `awk` works, we'll print the record and field number before each field.

```
awk '{ for (i = 1; i <= NF; i++)
       print NR":"i, $i } ' $*
```

Notice that the print statement concatenates NR, a colon, and `i`. The comma produces an output field separator, which is a blank by default.

This program produces the following results on a sample file:

```
1:1 awk
1:2 Aho,
1:3 Weinstein
1:4 &
1:5 Kernighan
2:1 BASIC
```

```
2:2 Beginner’ s
2:3 All-Purpose
2:4 Symbolic
2:5 Instruction
2:6 Code
```

`Symbolic` is the fourth field of the second record. You might note that the sample file is `acronyms`, the one in which we inserted a tab character between the first and second fields. Because we did not change the default field separator, `awk` interpreted the tab or blank as a field separator. This allows you to write programs in which the special value of the tab is ignored.

Conditional Statements

Now let’s change our example so that when given an argument, the program returns the record and field number where that argument appears.

Essentially, we want to test each field to see if it matches the argument; if it does, we want to print the record and field number. We need to introduce another flow control construct, the `if` statement. The `if` statement evaluates an expression—if true, it performs the procedure; if false, it does not.
In the next example, we use the `if` statement to test whether the current field is equal to the argument. If it is, the current record and field number are printed.

```
awk '{ for (i = 1; i <= NF; i++) {
      if ($i == search) {
             print NR":"i
             }
      }
} ' search=$1 acronyms
```

This new procedure prints *2:1* or *3.4* and isn’t very useful by itself, but it demonstrates that you can retrieve and test any single field from any record.
The next program, `double`, checks if the first word on a line is a duplicate of the last word on the previous line. We use `double` in proofing documents and it catches a surprisingly common typing mistake.

```
awk '
NF > 0 (
      if ($1 == lastword){
            print NR ": double " $1
            }
            lastword = $NF
}' $1
```

When the first line of input is read, if the number of fields is greater than 0, then the expression in the `if` statement is evaluated. Because the variable `lastword` has not been set, it evaluates to false. The final action assigns the value of `$NF` to the variable `lastword`. (`$NF` refers to the last field; the value of `NF` is the number of the last field.) When the next input line is read, the first word is compared against the value of `lastword`. If they are the same, a message is printed.

```
double sect1
15: double the
32: double a
```